TECHNICAL REVIEW · FOR THE ENGINEERING TEAM

BMS Industrial IoT Platform

Current architecture, the technology in use, the guarantees the system makes, and what is still outstanding before production. Every statement here was read from the repository.

6	2	.NET 10	Next.js 16	120+
RUNTIME SERVICES	DATABASES	BACKEND	DASHBOARD	AUTOMATED TESTS

What the platform does

Field devices publish readings over MQTT. The platform decodes each vendor's payload into a common shape, stores the original bytes and the normalized reading, evaluates alert rules and derived values, and pushes live updates to a browser dashboard.

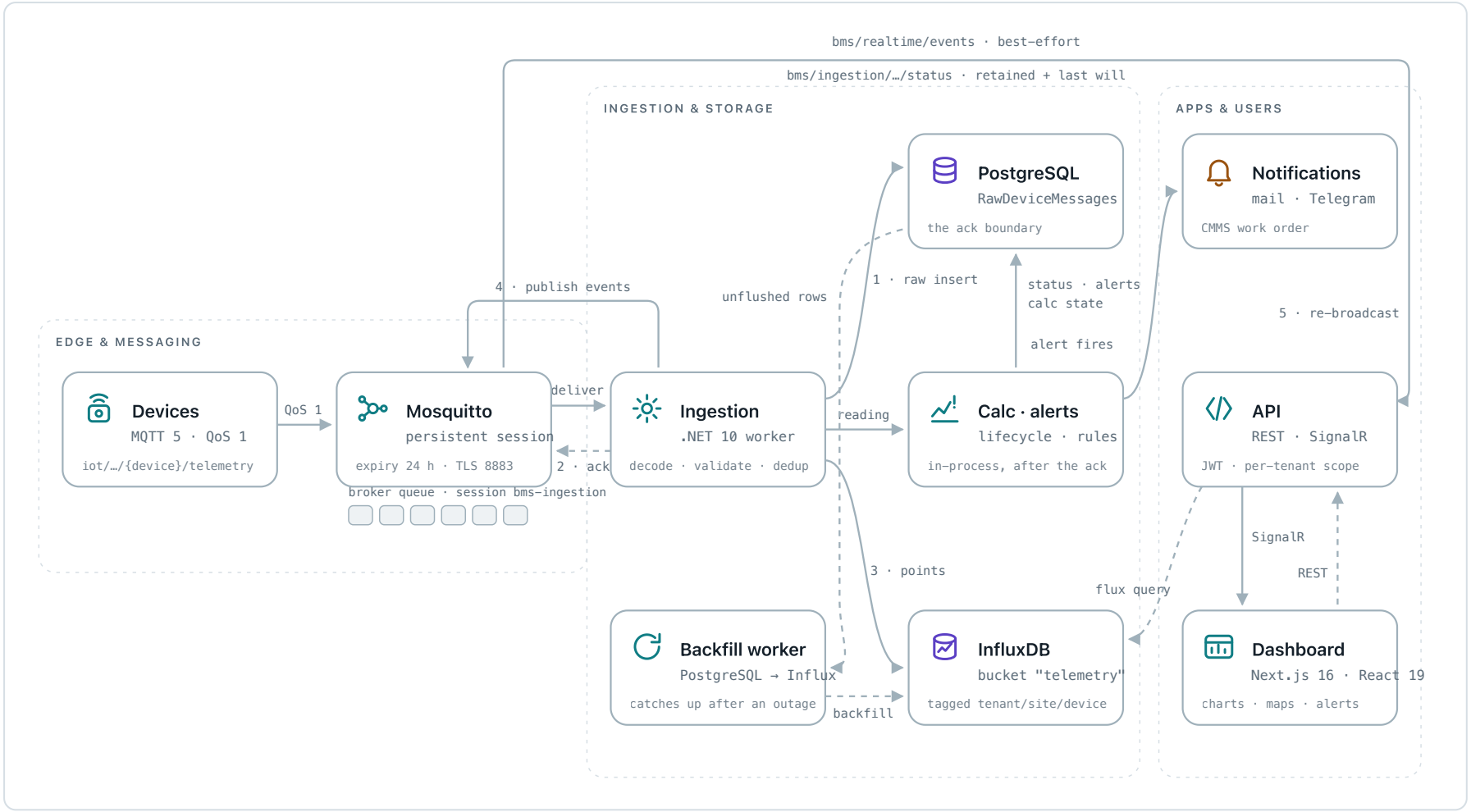
- **Multi-tenant.** Every device, reading and alert belongs to a tenant and a site; access is scoped by JWT claims and enforced again at the broker by per-tenant MQTT ACLs.
- **Vendor-agnostic.** JSON, binary, hex and BACnet payloads are decoded by pluggable decoders and mapped to internal field names by versioned payload mappings.
- **Never discards raw data.** The exact bytes received are kept, so a vendor's malformed message can be debugged and replayed after a decoder fix.
- **Survives its own outages.** Each dependency can fail independently without losing a reading. Section 5 states exactly what is guaranteed.

The journey of one reading

1	Device publishes to `iot/{tenant}/{site}/{device}/telemetry` at QoS 1 over TLS.
2	Mosquitto delivers it to the `bms-ingestion` persistent session.
3	The worker decodes, validates and deduplicates it *in memory*.
4	**The raw bytes are committed to PostgreSQL.**
5	**Only now is the broker acknowledged.** Anything that failed before this point is redelivered.
6	The normalized point is written to InfluxDB.
7	Device status, calculations and alert rules run; alerts go to email, Telegram and CMMS.
8	A live event is published back to the broker; the API re-broadcasts it over SignalR to the dashboard.

Steps 4 and 5 in that order are the core design decision. Everything in section 5 follows from it.

1 · Architecture



Teal arrows carry telemetry forward, violet carry acknowledgements and read-backs, amber are repair paths. The numbered arrows are the write order: raw insert, then the acknowledgement, then the time-series point, then the live event, then the re-broadcast. An interactive version of this diagram, with four failure scenarios animated, is at `docs/architecture/BMS Live Architecture.html`.

2 · Services and what each one owns

SERVICE	RUNS AS	RESPONSIBILITY	FAILS HOW
Mosquitto	container	MQTT 5 broker. Device authentication and per-tenant ACLs via mosquitto-go-auth against PostgreSQL. Holds the persistent `bms-ingestion` session and queues QoS 1 messages while the worker is away.	Devices cannot publish; they must buffer or drop. This is the only hop the platform cannot cover.
Bms.Ingestion	container (own process)	MQTT and TCP workers, the ingestion pipeline, device lifecycle, calculations, alert evaluation, notification dispatch, the InfluxDB backfill worker and the liveness publisher.	Broker queues everything; nothing is lost up to the queue limits. Its last will tells the API it is gone.
Bms.Api	container	REST API, JWT auth, SignalR hub, the real-time bridge subscriber, offline and timeout sweeps, Swagger.	Dashboard goes dark. Ingestion and storage continue unaffected.
PostgreSQL 17	container + volume	Tenants, sites, devices, device types, payload mappings, alert rules and alerts, calculation rules and state, users, MQTT credentials, and every raw device message.	Messages are not acknowledged and are redelivered. Device logins are also refused, since broker auth reads the same database.
InfluxDB 2	container + volume	The `telemetry` bucket: normalized readings tagged by tenant, site and device; the source for charts, history and reports.	Readings are already safe in PostgreSQL. The backfill worker replays them when it returns. Charts lag.
Dashboard	container	Next.js app: live telemetry, charts, site maps, device management, alerts, role-based navigation.	No impact on data capture.

Why ingestion is a separate process

Ingestion must keep draining the broker whether or not anyone is using the API, and it must be restartable and scalable on its own schedule. Splitting it costs one thing: real-time events generated there cannot reach the API's in-memory SignalR hub directly.

Rather than introduce Redis or another backplane for that one hop, the worker publishes events to an internal topic on the broker the platform already runs — `bms/realtime/events` — and the API subscribes and re-broadcasts them. The same hop carries the retained ingestion status topic and the worker's MQTT last will, which is how the dashboard distinguishes "ingestion is down" from "every device went quiet".

> **Trade-off accepted:** that channel is best-effort — it is skipped while the broker is disconnected and its failures are logged and dropped. PostgreSQL and InfluxDB, not the event channel, are the source of truth. A missed event costs UI freshness, never data.

3 · Technology stack

BACKEND — .NET 10 (NET10.0)	
ASP.NET Core	10.0.11 — REST API, JWT bearer auth, Swagger, health checks
SignalR	10 — live telemetry and alerts to browsers
MQTTnet	5.2.0.1603 — MQTT 5 client, manual acknowledgement
EF Core + Npgsql	10.0.11 / 10.0.3 — PostgreSQL access, migrations, retry on failure
InfluxDB.Client	5.1.0 — time-series writes and Flux queries
BCrypt.Net-Next	4.2.0 — password hashing (shared with broker auth)
JWT	8.22.0 — token issuance and validation

DATA	
PostgreSQL	17-alpine — operational data and raw messages
InfluxDB	2 — bucket `telemetry`

FRONTEND	
Next.js	16.3.4 — App Router, role-based navigation
React	19.2.8 with TypeScript 5
Tailwind CSS	4
Recharts	3.10.1 — telemetry charts
Leaflet	1.9.4 + react-leaflet 5 — site maps
@microsoft/signalr	10.0.11 — live updates

MESSAGING & DELIVERY	
Mosquitto	via `iegomez/mosquitto-go-auth` — MQTT 3.1.1 / 5, TLS listener on 8883, plain on 1883
Auth backend	Device logins from PostgreSQL, service accounts from file, per-tenant ACL
Docker Compose	6 services, restart policies, health checks, named volumes
Tests	xUnit + Moq — 120+ unit tests, WebApplicationFactory integration tests, plus a bash reliability suite that runs real outages against an isolated stack
Metrics	`System.Diagnostics.Metrics` counters in Bms.Ingestion

Code structure — dependencies point inward · no business rule references MQTT, EF Core, InfluxDB or Next.js

Bms.Domain	Entities and rules. No dependencies at all.
Bms.Application	Use cases (the ingestion pipeline, normalizer, backfill) and the interfaces every outer project implements.
Bms.Infrastructure	EF Core repositories, InfluxDB store, email / Telegram / CMMS senders, firmware storage.
Bms.Protocols	Transports (MQTT, TCP, HTTP), decoders (JSON, binary, hex, BACnet), payload mappers.
Alerting · Calculations · DeviceManagement	Alert rule evaluation, derived values, device online/offline lifecycle.
Bms.Api · Bms.Ingestion	The two deployable hosts. Composition roots only.

4 · What the system guarantees, and why

PostgreSQL commits before the acknowledgement

The worker uses manual acknowledgement. The raw row commits first; only then is the broker told the message is handled. A message that fails anywhere before that point is never acknowledged, so the broker redelivers it after a retry or a reconnect.

InfluxDB is written after the acknowledgement

A failed point write therefore never fails a message. The row stays marked unflushed and the backfill worker replays it — with exponential backoff while InfluxDB is unreachable, and immediately again while a backlog is draining.

Redelivery is safe

A duplicate key is built from what the device itself sent — sequence number and device timestamp — so a redelivered copy produces the same key while a genuinely new reading does not. A unique index on `(DeviceId, DedupKey)` settles any race the check loses.

A bad message cannot stop the queue

A failure that is about the message itself is recorded as `Rejected` with the reason and acknowledged, rather than blocking everything behind it. Only infrastructure failures are retried.

Downstream effects are isolated

An InfluxDB outage must not also stop device-status tracking or alert evaluation — arguably more important during an outage than the chart. Each effect catches its own failures. The same applies per notification channel.

Nothing is discarded

Rejected and duplicate messages are kept with their reason. The original bytes are always preserved, which is what makes replay after a decoder fix, and proving what a vendor actually sent, possible.

5 · Behaviour when something is down

FAILURE	WHAT HAPPENS	DATA OUTCOME	BOUNDED BY
Ingestion stopped, crashed or deploying	Mosquitto keeps the session and its subscriptions and queues every QoS 1 message. The retained last will tells the API the worker is gone.	**No loss** — the backlog drains in order on reconnect.	The queue limits and the 24 h session expiry. see action 2
PostgreSQL down	Decoding fails before the insert is attempted, so nothing is acknowledged. The worker retries the same message with exponential backoff while the rest queue behind it.	**No loss** — redelivered copies are absorbed by the dedup index when it returns.	Broker auth reads the same database, so devices that reconnect during the outage are refused and must buffer themselves. see action 4
InfluxDB down	The message is already committed and acknowledged. The point write fails in isolation and the row is marked unflushed.	**No loss** — the backfill worker replays it.	Charts and reports lag until the replay completes.
Mosquitto crashed or killed	Queue restored from the last autosave on restart.	Up to **60 seconds** of queued messages can be lost.	`autosave_interval`, currently 60 s.
Broker unreachable from the device	The device cannot publish at all.	**Loss is possible** — it depends entirely on whether the device buffers.	Device firmware. The only hop the platform cannot cover.
API down	Dashboard goes dark; capture and alerting continue.	**No loss.**	Notification channels are driven by ingestion, not the API, so alerts still go out.

MQTT configuration in force

Protocol	MQTT 5	**Clean start**	`false` (session resumed)	**Session expiry**	86,400 s (24 h)	**Receive maximum**	100 in flight
Telemetry QoS	1 (at least once)	**Queue limit**	100,000 msgs / 100 MiB	**Autosave**	60 s	**Last will**	retained, on the status topic

6 · Decisions taken, and the conditions to revisit them

Each of these was a deliberate choice to not add infrastructure. None is permanent; each has a stated trigger. The point of writing them down is so the next person does not re-litigate them from scratch, and does not adopt them out of habit either.

NOT USED	WHAT IT WOULD SOLVE	WHY IT IS NOT NEEDED HERE	ADOPT IT WHEN
Kafka	A durable, replayable log with many independent consumers, partitioned for horizontal scale.	Devices speak MQTT only, so Kafka could not replace Mosquitto — it would be added behind it, as a second broker and an extra hop. The durable log already exists as the raw message table, and the backfill worker already replays from it. There is exactly one consumer today. A single-broker Kafka on one host also runs at replication factor 1, which is weaker than the PostgreSQL commit the system currently trusts.	Two or more of: one ingestion process provably cannot keep up in normal operation; three or more jobs are scanning the raw table on their own schedules; reprocessing days of raw data across the fleet is too slow in PostgreSQL; the required buffer is hours rather than minutes; an external party needs a live feed. And someone owns operating it.
Redis or another backplane	Pub/sub between the ingestion process and the API's SignalR hub.	One internal MQTT topic on the broker already running does the same job with no new service, no new port and no new failure mode.	The event volume or the delivery guarantees outgrow a best-effort channel — for example if a missed UI event ever becomes a correctness problem rather than a freshness one.
Kubernetes	Multi-node orchestration for independently scaled services.	Six containers on one host with restart policies and health checks. There is no fleet to orchestrate.	Multiple nodes, independent scaling per service, or zero-downtime rolling deploys become a requirement.
Microservices	Independent deployability and scaling boundaries.	The boundaries that matter — transport, protocol, payload format, mapping, domain versus time-series storage — are already enforced in-process by Clean Architecture. Splitting them over the network would add latency and distributed failure modes for no benefit at this size.	A specific boundary genuinely needs to scale or ship separately. Ingestion was split out for exactly that reason and is the precedent to follow.

7 · What we still need before production

The architecture is sound and the failure behaviour is tested. What follows is not redesign work — it is the operational gap between "it works on the stack" and "it runs unattended with real devices and real money on it". Items marked must should block go-live.

> **The cheaper moves to try before any of the above.** In order: batch the InfluxDB writes instead of one point per message; enable MQTT 5 shared subscriptions and run two or three ingestion instances; time-partition the raw message table; then re-measure. Each is days of work rather than a new platform to operate, and together they move the ceiling a long way.

	ITEM	WHY IT MATTERS	WHAT DONE LOOKS LIKE
must	**Raw message retention**	There is no retention or cleanup worker in the codebase today. `RawDeviceMessages` stores the full payload of every message forever. At the target device count this fills the disk, and a full disk stops the acknowledgement path — which is the one thing that must never stop.	An agreed retention window, a worker that enforces it, and the table time-partitioned so deletes are a partition drop rather than a mass delete.
must	**Size the broker queue against real numbers**	The current limits are 100,000 messages / 100 MiB. At 10,000 devices reporting once a minute that is roughly **10 minutes** of outage cover; at once every 10 seconds it is roughly **60 seconds**. Beyond the limit Mosquitto drops silently. The 24-hour session expiry is not the real bound — the queue is.	The sizing formula in `docs/mqtt/reliability.md` applied to the real device count and interval, the limits set accordingly, and the volume sized for the bytes.
must	**Real secrets and real certificates**	`.env.example` ships `change-me` placeholders for the database, InfluxDB token and all MQTT accounts. The TLS listener needs a certificate that devices will actually trust.	Generated credentials held in a secret store with a rotation plan, a real CA and server certificate on 8883, and anonymous access confirmed off.
must	**Decide the PostgreSQL coupling posture**	PostgreSQL is both the acknowledgement boundary and the broker's device-authentication backend. During a PostgreSQL outage, devices that try to reconnect are **refused**, so an outage does not only pause processing — it can push devices off the broker entirely.	A conscious decision: cache authentication lookups, use a read replica for auth, or accept it and tell device vendors what buffering they must do. Any of the three is fine. Not knowing is not.

7 · What we still need before production · continued

	ITEM	WHY IT MATTERS	WHAT DONE LOOKS LIKE
must	**Backups and a restore drill**	Three stateful volumes — `pgdata`, `influxdata`, `mosquitto-data`. A backup that has never been restored is not a backup.	Scheduled backups for both databases, off-host, plus one rehearsed restore with the time it took written down.
must	**Monitoring and alerting**	The counters already exist — processing latency, messages persisted / rejected / duplicate, transient failures, time-series write failures — but nothing watches them. Without that, an outage lasts until a user complains.	Those metrics plus broker queue depth and the ingestion status topic on a dashboard, with alerts on queue depth rising, rejection rate and the worker going offline.
scale	**Shared subscriptions for more than one ingestion instance**	Ingestion runs under a single client id with a persistent session, so a second instance cannot be added — it would fight for the same session. This is a hard ceiling, and it is the first one you will hit.	MQTT 5 shared subscriptions (`$share/...`) proven on the stack, so scaling out is a configuration change rather than a redesign under pressure.
scale	**Batch the InfluxDB writes**	One point write per message is the simplest thing that works and the first thing to become a bottleneck.	Buffered writes with a size and time trigger, with the unflushed marking still correct so backfill keeps working.
scale	**Load test at the target device count**	Every number in this document about capacity is an estimate. One test replaces all of them with facts, and sizes items 2, 7 and 8 properly.	A simulated run at the real device count and interval, with processing latency and queue depth recorded.
tidy	**One code comment contradicts the code**	`MqttRealtimeNotifier` documents the bridge as QoS 0, but it publishes through an adapter that hardcodes QoS 1. Harmless today; misleading to whoever reads it next.	Either correct the comment or make the publish QoS a parameter, depending on which behaviour was intended.

Appendix · reference

MQTT topics in use

TOPIC	DIRECTION	PURPOSE
iot/{tenant}/{site}/{device}/telemetry	device → platform	Readings, QoS 1
iot/{tenant}/{site}/{device}/heartbeat	device → platform	Liveness
iot/{tenant}/{site}/{device}/commands/ack	device → platform	Command acknowledgement
iot/{tenant}/{site}/{device}/commands	platform → device	Downlink commands
iot/{tenant}/{site}/{device}/config	platform → device	Configuration push
iot/{tenant}/{site}/{device}/firmware	platform → device	OTA deployment
bms/realtime/events	ingestion → API	Internal bridge to SignalR
bms/ingestion/{id}/status	ingestion → API	Retained liveness + last will

Access is enforced per account in the broker ACL: devices are confined to their own tenant subtree by pattern, the ingestion account reads all of `iot/#`, and the API account may only read events and status and write the three downlink topics.

Where things live

src/backend/	Nine .NET projects; two of them are deployable hosts
src/frontend/bms-dashboard/	Next.js dashboard
infrastructure/mosquitto/	Broker config template, ACL, entrypoint
infrastructure/docker/	API and ingestion Dockerfiles
tests/	Unit, integration, and a bash reliability suite that runs real outages
docs/architecture/	System architecture, data flow, and the interactive diagram
docs/mqtt/	QoS and reliability, including the queue sizing formula
docs/deployment/	Docker and deployment notes

Reading this further

- `docs/architecture/BMS Live Architecture.html` — the diagram on page 2, animated, with the four failure scenarios playable.
- `docs/mqtt/reliability.md` — the queue sizing formula and the full failure matrix.
- `docs/architecture/system-architecture.md` — the longer form of section 6.

> **One sentence to take away.** The platform's reliability rests on a single ordering rule — commit the raw bytes to PostgreSQL, then acknowledge the broker, then do everything else. Every guarantee in section 4 follows from it, and any change to the ingestion pipeline should be checked against it first.